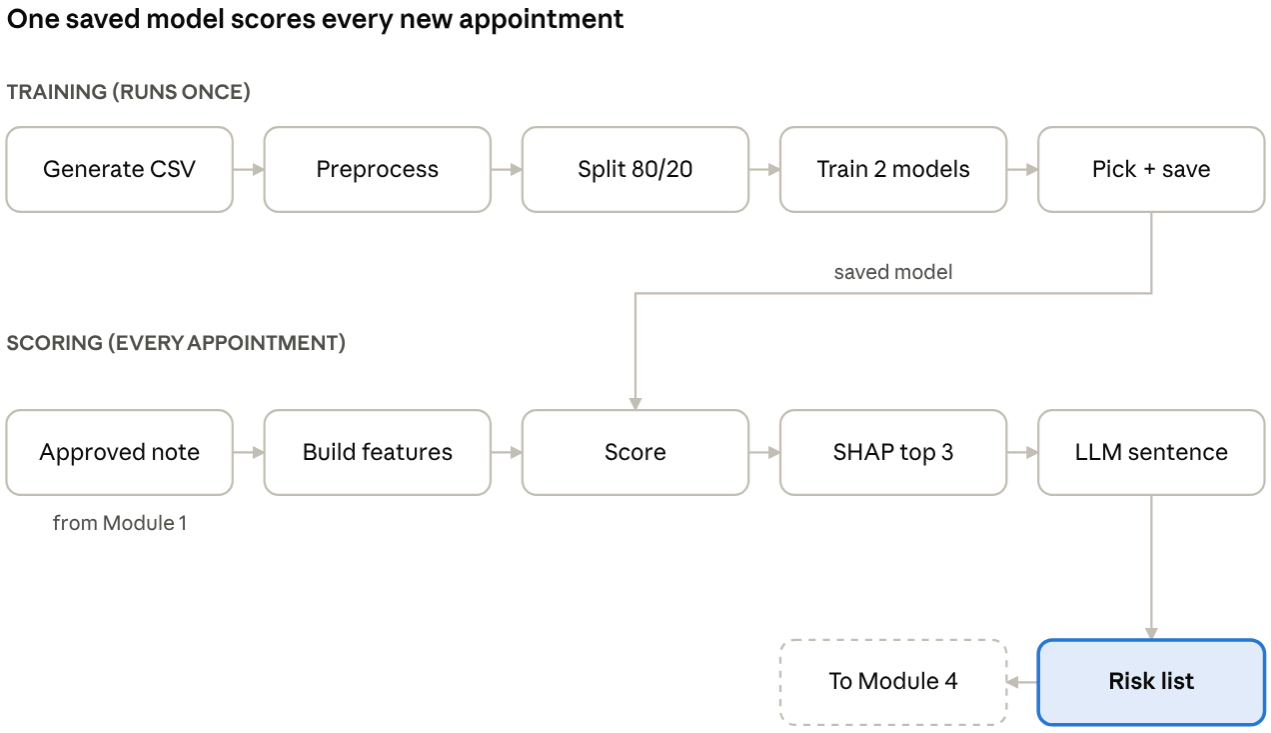
MediFlow AI — Module 3 Architecture (Risk Engine)

Module 3 predicts which patients will miss their follow-up appointment, gives each appointment a risk score with a plain-language reason, and hands the high-risk list to Module 4 for reminders. Each section below can be handed to a coding agent as-is.

Flow at a glance



Module 3 flow · training + scoring

Training builds and saves one model. Every appointment created from an approved Module 1 note is then scored by that saved model, explained with SHAP and one LLM sentence, and placed on the risk list Module 4 reads.

Role in the product

Module 3 decides *who needs a reminder*. A missed follow-up means an unfinished treatment for the patient and an empty slot for the clinic; this module finds those patients before it happens.

Scope for the prototype: one model, no-show prediction. Readmission risk is left for a later version because no-show links directly to Module 4 and is easy to explain and measure.

Connections

- **From Module 1:** when a doctor approves a note with a follow_up_date, an appointment is created. That appointment is scored automatically.
- **To Module 4:** the scored list, sorted by risk, is what the Patient messenger uses to decide who gets a reminder and how many.
- **To Module 5:** the Care coordinator calls the scoring endpoint as a tool when planning a patient's next steps.
- **On screen:** a "Upcoming appointments" page shows each appointment with its risk band and a one-sentence reason.

Target and features

Target variable: `no_show` : 1 if the patient missed the appointment, 0 if they attended. This is binary classification. The model outputs a risk score from 0 to 1 for each appointment.

Features: 10, all known *before* the appointment day.

Feature	Type	Range / values	Why it is included
age	integer	15–85	Young adults miss more; older chronic patients are more regular
sex	category	female, male	Included to test fairness: it should carry little or no weight
distance_km	number	0.5–60	Travel cost and time make attending harder
nhis_active	yes/no	1, 0	Without active insurance, cost keeps people away
previous_visits	integer	0–20	Regular patients are more reliable
previous_no_shows	integer	0 to previous_visits	Strongest predictor: past behaviour repeats

Feature	Type	Range / values	Why it is included
lead_time_days	integer	1–60	Longer gap between booking and appointment means more forgetting
day_of_week	category	Mon–Sat	Mondays and market days see more misses
rainy_season	yes/no	1, 0 (from the appointment month: Apr–Jul, Sep–Nov)	Rain makes travel harder
visit_type	category	follow_up, chronic_review, antenatal	Acute patients feel better and skip; antenatal patients rarely miss

Columns in the CSV that are NOT features: appointment_id, patient_id, appointment_date. They identify rows; feeding them to the model would let it memorise rather than learn.

Leakage rule: a feature is only allowed if it is known when the appointment is booked. Anything known only on or after the day (cancellation calls, check-in time, rescheduled flag) is excluded, because it gives away the answer and makes the model look better in testing than it would be in real use.

Synthetic data design

The data is generated by a seeded Python script, not an LLM: an LLM cannot produce 5,000 rows with consistent statistical relationships, but a script can. Output is a clean CSV, no missing values.

How the target is created

1. Generate the 10 features for each row from realistic distributions.
2. Compute a hidden risk number (log-odds) from the features using fixed rules (below).
3. Add random noise, so the model can learn the pattern but never reach a perfect score.
4. Turn it into a probability (sigmoid) and draw `no_show` as 1 or 0 from that probability.
5. Adjust the base value so about 20% of rows are no-shows.

Hidden rules (written to `data/ml/ground_truth_rules.md` so you can compare what the model learned against the truth)

Feature	Effect on no-show risk
previous_no_shows / previous_visits	Strong increase
distance_km	Moderate increase
lead_time_days	Moderate increase
nhis_active	Moderate decrease
visit_type	follow_up increases; antenatal strongly decreases; chronic_review slightly decreases
rainy_season	Small increase
day_of_week	Monday and Friday small increase
age	18–35 small increase; 60+ small decrease
sex	No effect
noise	Random, normal, standard deviation 0.5 on the log-odds

Why this design matters in interviews: because the true rules are known, you can show that the model's SHAP importance matches them, and that `sex` correctly ends up near zero. That proves the pipeline learns real signal, not noise.

Training pipeline

Runs once with `python ml/train_no_show.py`, and again whenever the data changes. Random seed 42 everywhere so results are reproducible.

C1. Load and check

- Load `data/ml/appointments.csv`.
- Check: expected columns present, no missing values, values inside the ranges in *Target and features*, `previous_no_shows` never above `previous_visits`, no-show rate between 15% and 25%. Stop with a clear error if any check fails.

C2. Preprocess

- One-hot encode `sex`, `day_of_week`, `visit_type` (one column per value).
- Scale numeric features with StandardScaler **for logistic regression only**. XGBoost uses trees, which split on thresholds and don't care about scale.

- Wrap preprocessing and model together in one scikit-learn `Pipeline` with a `ColumnTransformer`. **Why:** the exact same transformation is then applied at scoring time, so a new appointment is never encoded differently from the training data.

C3. Split

- 80% train, 20% test, `stratify=no_show`. **Why stratify:** both sets keep the same ~20% no-show ratio, so the test score is fair.
- The test set is not touched until final evaluation.

C4. Baseline: logistic regression

- `LogisticRegression(class_weight="balanced", max_iter=1000)`.
- **Why a baseline:** it is simple and explainable, and it sets the score XGBoost must beat. If the complex model can't beat the simple one, the complexity isn't worth it.

C5. Main model: XGBoost

- `XGBClassifier` with `scale_pos_weight = (number of 0s) / (number of 1s)` in the training set, about 4.
- **Why scale_pos_weight:** with only 20% no-shows, a model can reach 80% accuracy by predicting "everyone comes" and catch zero no-shows. Weighting each no-show about 4 times makes missing one cost as much as the whole majority, so the model actually learns to find them. It is XGBoost's equivalent of `class_weight="balanced"`.
- **Small tuning grid** with 5-fold stratified cross-validation on the training set, scored by AUC: `max_depth` 3, 4, 5; `n_estimators` 200, 400; `learning_rate` 0.05, 0.1. That's 12 combinations, which is enough for a prototype.
- **Why XGBoost:** it is the strongest standard choice for table-shaped data, handles mixed feature types, and works directly with SHAP for explanations. Random forest is skipped because XGBoost usually does the same job better.

C6. Evaluate and pick

- Score both models on the test set: AUC, precision, recall, confusion matrix.
- Ship XGBoost only if its AUC beats the baseline's. If it doesn't, ship the baseline and note it in the model card.
- **Accuracy is not used to choose,** because on imbalanced data it rewards predicting "everyone comes".

C7. Save

- Save the whole pipeline to `models/no_show/v1/model.joblib`.
- Save `metrics.json`, the feature list, and plots (ROC curve, precision-recall curve, confusion matrix, SHAP summary) to the same folder.
- Write `docs/model-cards/no_show_v1.md`: purpose, data, features, metrics, limits.

A note on the score: because scale_pos_weight boosts no-shows during training, the output scores are good for *ranking* patients but are not exact probabilities. Calibrating them (`CalibratedClassifierCV`) is listed as a later improvement in the model card.

Scoring and explanation

Runs every time an appointment is created, and once a day for all upcoming appointments.

C8. Build the feature row

- **Trigger:** a Module 1 note is approved with a follow_up_date, which creates a row in `appointments`.
- **Job:** assemble the 10 features for that appointment: age, sex, distance and NHIS status from the patient record; previous_visits and previous_no_shows from appointment history; lead_time_days, day_of_week, rainy_season and visit_type from the appointment itself.

C9. Score

- Load the saved pipeline once at startup and call `predict_proba`.
- **Bands:** High ⩾ 0.50, Medium 0.25–0.49, Low < 0.25.

C10. Explain with SHAP

- Use `shap.TreeExplainer` on the XGBoost model.
- Take the 3 features with the largest positive SHAP values (the ones pushing the risk *up*), with their actual values. Example: `previous_no_shows=2 (+0.9)`, `distance_km=38 (+0.5)`, `lead_time_days=30 (+0.3)`.

C11. Turn it into one sentence (LLM)

Claude writes the sentence the nurse sees. Prompt stored as `prompts/risk_explain_v1.txt`:

```
Write one sentence (under 30 words) for a clinic nurse explaining why
this appointment has a {band} risk of being missed.

Use ONLY these factors, in this order of importance. Do not add any
other reason, and do not mention numbers like SHAP values.
{top_factors_with_values}

Use plain words: "has missed 2 of her last 4 visits",
"lives about 38 km away", "was booked 30 days ahead".
Do not give medical advice. Do not use the patient's name.
```

- **Fallback:** if the LLM fails or the sentence mentions a factor not in the list, use a template instead: "Main risk factors: missed visits, distance, long wait."

C12. Save and publish

- Save the score, band, top factors, sentence and model version to `risk_scores`.
- The Upcoming appointments page lists them sorted by score, highest first. This list is what Module 4 reads.

Guardrails

The rule behind every guardrail: a risk score is a prompt to help a patient, never a reason to deny one care.

Guardrail	Rule	If it fails
Data checks (C1)	Columns, ranges, no missing values, no-show rate 15–25%	Training stops with a clear error
Leakage check	Feature list in code must equal the approved list of 10	Training stops
Beat the baseline	XGBoost test AUC > logistic regression test AUC	Ship the baseline; note it in the model card
Fairness check	`sex` ranks in the bottom 3 by mean SHAP importance	Flag in the model card for review
Scoring input check	Every feature present and in range before scoring	Appointment marked "not scored" with the reason
Score range	Output between 0 and 1	Logged as an error; not shown
Explanation check	Sentence names only the 3 provided factors and no patient name	Use the template sentence
Use of the score	Score only decides reminder priority; it never blocks booking or treatment	Stated on the page and in the model card

Storage, tech and API

`appointments`: appointment_id, patient_id, appointment_date, booked_on, visit_type, source_note_id (the Module 1 note that created it), attended (filled after the day; used for retraining later).

`risk_scores`

Column	What it holds
appointment_id	Which appointment
score, band	e.g. 0.71, High
top_factors	The 3 SHAP factors with values (JSON)
explanation	The one-sentence reason
model_version	e.g. `no_show_v1`
scored_at	Timestamp

Tech

Part	Choice
Data generation	Python, numpy, pandas
Modelling	scikit-learn (Pipeline, ColumnTransformer, LogisticRegression, GridSearchCV), xgboost
Explanations	shap
Saving models	joblib, one folder per version
Plots	matplotlib
Experiment tracking	MLflow (optional for the prototype)

API endpoints

- `POST /risk/score/{appointment_id}`: score one appointment and save it.
- `POST /risk/score-upcoming`: score all appointments in the next 30 days (the daily job).
- `GET /risk/upcoming`: list upcoming appointments sorted by score, with band and sentence.
- `GET /risk/model-card`: model version, metrics and plots.

KPIs and evaluation

All KPIs are measured on the 20% test set (about 1,000 appointments) and written to `metrics.json` and the model card.

KPI	Plain meaning	Target
AUC (ROC)	How well the model ranks people who miss above people who come. 0.5 = coin flip, 1.0 = perfect	⩾ 0.75
Precision (High band)	Of patients flagged High, how many actually missed	⩾ 50%
Recall (High band)	Of all patients who missed, how many were flagged High	⩾ 60%
Beats baseline	XGBoost AUC minus logistic regression AUC	> 0
Matches the hidden rules	Top 3 SHAP features are among previous_no_shows, visit_type, distance_km, lead_time_days, nhis_active	Yes
No-shows caught (business KPI)	Number of actual no-shows in the test set that land in the High band: the patients Module 4 would remind	Reported as a count and %

Precision vs recall trade-off: missing a no-show (low recall) costs a lost visit; a wasted reminder (low precision) costs one SMS. So recall matters slightly more here, which is why its target is higher.

Interview line: on synthetic data these scores reflect the rules we built. The point is the pipeline: on real hospital data it retrains unchanged, and the numbers become a true measure.

Definition of done

Module 3 is done when all of these work:

- [ ] The generator creates `appointments.csv` (5,000 rows) and `upcoming_appointments.csv`, and prints the summary tables
- [ ] `python ml/train_no_show.py` trains both models and saves the pipeline, metrics, plots and model card

- [ ] XGBoost beats the baseline and meets the AUC, precision and recall targets, or the gap is explained in the model card
- [ ] SHAP top features match the hidden rules and `sex` is near zero
- [ ] Approving a Module 1 note with a follow-up date creates an appointment that is scored automatically
- [ ] The Upcoming appointments page lists the 10 demo patients with band and one-sentence reason
- [ ] Every score is saved in `risk_scores` with its model version

Synthetic data prompt

Give this to your coding agent. Run it after the Module 1 transcripts exist, so the 10 demo patients are included.

```
Write data/generators/generate_appointments.py, a seeded (seed 42)
Python script using numpy and pandas. Do NOT use an LLM for this data.

Output 1: data/ml/appointments.csv, 5,000 past appointments, no missing
values, columns:
  appointment_id (A00001...), patient_id, appointment_date (2024-2026),
  age, sex, distance_km, nhis_active, previous_visits,
  previous_no_shows, lead_time_days, day_of_week, rainy_season,
  visit_type, no_show

Feature distributions:
- age 15-85 (antenatal rows: 16-45 and female only)
- sex female/male, about 60/40
- distance_km: right-skewed, most 1-15, some up to 60
- nhis_active: about 70% = 1
- previous_visits 0-20; previous_no_shows 0 to previous_visits
- lead_time_days 1-60, most 7-21
- day_of_week Mon-Sat (no Sunday)
- rainy_season = 1 if appointment month is Apr-Jul or Sep-Nov
- visit_type: follow_up 45%, chronic_review 35%, antenatal 20%
- About 2,000 unique patient IDs (MF-00011 onward), so patients repeat

Create the target like this:
1. log_odds = base
   + 2.0 * (previous_no_shows / max(previous_visits, 1))
   + 0.03 * distance_km
   + 0.02 * lead_time_days
   - 0.6 * nhis_active
   + 0.5 if visit_type == follow_up
   - 1.2 if visit_type == antenatal
```

```
   - 0.2 if visit_type == chronic_review
   + 0.3 * rainy_season
   + 0.25 if day_of_week in (Mon, Fri)
   + 0.2 if age 18-35;  - 0.3 if age >= 60
   + 0 for sex (no effect)
   + noise ~ Normal(0, 0.5)
2. p = sigmoid(log_odds); no_show = Bernoulli(p)
3. Tune `base` so the overall no-show rate is 19-21%.

Output 2: data/ml/upcoming_appointments.csv, one future follow-up
appointment for each of the 10 Module 1 patients (MF-00001 to MF-00010;
read age and sex from data/transcripts/), same columns but no no_show.
Give them a mix of histories so the demo shows High, Medium and Low.
Also add their past appointments to appointments.csv so their
previous_visits and previous_no_shows are consistent.

Output 3: data/ml/ground_truth_rules.md, the formula above in plain
words.

Finally print: row count, overall no-show rate, no-show rate by
visit_type, by previous_no_shows (0, 1, 2+), by distance band
(<5, 5-20, 20+ km), and by sex.
```

When it's done, check the printed summary yourself: antenatal should have the lowest no-show rate, patients with 2+ past no-shows the highest, and male vs female should be roughly equal.